Hermes: Natural Language to SPARQL as Planning over the SPARQL Algebra

Lam Nguyen
Case Western Reserve University
Cleveland, Ohio, USA

ABSTRACT

Text-to-query systems usually emit a whole query, then repair it. Hermes treats the SPARQL 1.1 algebra as the action space instead. A plan grows by one operator at a time, and execution on the graph after each operator reports the cardinality, how that cardinality moved, a sample of rows, and any error. A kernel in the style of Lean checks scoping, aggregation, and schema types before a plan is allowed to run, and it refuses to finish while a goal is still open. Prompt-only generation and a reimplementation of UniQGen sit in the same system, on the same graphs, behind a scorer that reproduces the published Text2SPARQL 2026 numbers and that also reports a strict variant and a reproducible subset. On the 2026 corporate track, with the local model `llama3.1`, the one-shot baseline reaches a set F_1 of 0.017 and UniQGen reaches 0.064. Both are far below the published leaderboard. These runs are a first measurement on one local model. The operator-level policies have not been measured yet.

KEYWORDS

SPARQL, knowledge graphs, natural language interfaces, query algebra, execution feedback

PVLDB Reference Format:
Lam Nguyen. Hermes: Planning over the SPARQL Algebra. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/lamng3/hermes.

1 INTRODUCTION

A question over a knowledge graph becomes useful once it is a query the graph can run. The usual pipeline asks a model for a complete SPARQL string and, if the string fails to parse or returns nothing, asks again [6, 11]. Query optimizers work at a finer grain. They apply one algebraic operator, estimate the effect, and keep or discard the step [3]. Hermes uses that grain for natural language to SPARQL. The operators of the SPARQL 1.1 algebra [10] are the actions. Execution on the graph is the observation after every action.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The same interface holds methods that were not written as planners. A one-shot prompt and UniQGen's Chase and Backchase [3, 11] are policies over this space. They share graph stores, a normalized suite of benchmarks, and one scorer, so a later comparison can hold the model and the call budget fixed. This paper records the design and the first finished runs. It does not answer the comparison. Four questions organize the rest of the work.

(1) Does operator-level execution feedback beat one-shot generation with repair, at equal model size and equal LLM-call budget?
(2) Which part of the observation matters: cardinality, the change in cardinality, row samples, or errors?
(3) Can a search policy over the same actions match a prompted policy with fewer model calls?
(4) How far do results on the Text2SPARQL corporate graph transfer to DBpedia and Wikidata?

The system is available at the artifact URL above, under CC BY 4.0. Benchmark files keep the licenses of their sources.

2 THE SPARQL ALGEBRA AS ACTIONS

A query is an immutable logical plan: triple patterns, filters, optional groups, unions, minus groups, bindings, grouping, aggregates, having-clauses, ordering, the projection, distinctness, limit, offset, and the query form (`SELECT` or `ASK`). Rendering the plan is deterministic. An action is a pure function from one plan to the next. Table 1 lists them against the SPARQL 1.1 algebra [10]. Two further actions read the graph and leave the plan unchanged. `lookup` finds entities whose label contains a string. `inspect` lists predicates around an entity or a class. `undo` restores the previous plan. `finish` stops the policy.

The explorer applies an action and runs the resulting plan. The observation records whether the query parsed, the solution count, the previous count when it changed, the projected columns, a sample of rows (five by default), and an error string when the step failed. Cardinality is a `COUNT(*)` over the rendered pattern; the sample is the same pattern under `LIMIT`. An `ASK` plan reports the Boolean in place of a count. `preview` returns that observation and does not commit the action, which is the planner's analogue of costing a rewrite before choosing it. A step that fails to parse, or that the kernel rejects (Section 3), is not committed and is not sent to the store.

Table 2 is the shape of that feedback, from a walk over a small example graph. It is an illustration, not a benchmark result.

Any policy can drive the explorer. The algebra agent is one such policy. Each turn it sees the question, the vocabulary, the current plan as SPARQL, and the observation after every earlier action, and it replies with one JSON action. The loop ends on `finish` or after 15 steps, and the result is the last plan that ran. Repeated actions

Table 1: Actions and the SPARQL 1.1 operators they apply.

Action	Algebra operator
`triple`	BGP, extended by join
`filter`	Filter
`optional`	LeftJoin
`union`	Union
`minus`	Minus
`bind`	Extend
`group`	Group
`aggregate`	Aggregation
`having`	Filter on groups
`order`	OrderBy
`project`	Project
`distinct`	Distinct
`limit`, `offset`	Slice
`ask`	ASK query form
`remove`	retract a triple or a filter

Table 2: Explorer feedback on a small example graph.

Action	Observation
`triple ?city ex:hosted ?g`	19 solutions
`triple ?city ex:country ex:USA`	19 → 8
`triple ?g ex:season "Winter"`	8 → 4
`distinct`	4 → 3

Table 3: The prover, read as a tactic language.

Lean	Hermes
Theorem statement	Goal: variables, kind, type, cardinality
Tactic	Algebra action
Local context Γ	Variable $\mapsto$ inferred classes
Kernel	`Kernel.check(plan, goal)`
Unknown identifier	Term outside the schema namespaces
Type mismatch	Domain, range, or a literal as subject
No goals left	No errors and no open goals

are rejected in the prompt so the model has to choose something else. UniQGen's Chase and Backchase, in these terms, is a beam search over `remove` and `triple`.

3 A KERNEL IN THE STYLE OF LEAN

The algebra agent will execute any plan that parses. The prover puts a kernel in front of that execution. The design follows interactive theorem provers such as Lean [2]: the question is a goal, algebra actions are tactics, and the kernel checks each step before the graph sees it. Table 3 is the correspondence.

The model states the goal first, for example one department, or a number, or a yes-or-no answer. The kind is `entity`, `literal`, `number`, or `boolean`. A class may be attached, and `many` is false when a single answer is required. Typing comes from the schema [9]. A triple `?x a C` gives `?x : C`. A triple `?x p ?y` gives `?x` the domain of `p` and `?y` its range. Classes are compared up to `rdfs:subClassOf`. Under the open-world assumption, a class conflict is a warning. Using a literal where an entity is required is an error, as is a literal in subject position, an unknown class or property in a namespace the schema covers, a `BIND` onto an already bound variable, an unbound variable in `FILTER`, `GROUP BY`, `HAVING`, `ORDER BY`, or `SELECT`, and a projected variable that is neither grouped nor aggregated.

Open goals are what `finish` still has to discharge: an `ASK` for a question that wants values, or the reverse; an answer variable that is not bound or not projected; a variable the goal calls a number that is not numeric; an entity variable typed as a literal; a required class that the variable's types do not fall under; extra projected variables; and, once the plan has run, more than one solution when the goal asked for one. `finish` is refused while any error or open goal remains. The prompt shows the proof state each turn: the typing context and the open goals. Ill-typed steps increment a rejection counter and never reach the store.

The prover is the algebra agent with this kernel installed. It has an implementation and tests. It does not yet have an experiment configuration, so Section 6 does not report it.

4 DESIGN

Hermes is a small runtime around that loop. A system is any callable from a question, a knowledge graph, and an optional context to a SPARQL string. The context can carry text, linked entities, and, for ablations only, reference answers. Four systems are registered: `training-free`, `uniqgen`, `algebra-agent`, and `prover`. Another package can add one through the `hermes.systems` entry point. Each receives the language model and the graph by construction, and experiment files pass constructor arguments (`beam_width`, `max_steps`, and so on) without code changes.

The one-shot system prompts a single draft from the ontology vocabulary, runs a fixed sequence of repairs over prefixes and labels, and, if the draft does not parse, makes one further model call with the parser error. UniQGen follows the three stages of Wang et al. [11], rendered to SPARQL on whatever store is configured. Entities are linked by label and their neighborhood is collected; one model call proposes a constraint table, and constraints with no match in the graph are pruned; Chase and Backchase then searches for a query that covers the reference answers and a minimal query that returns only those answers. Two choices differ from that paper, and they matter when reading Section 6. The query text is rendered from the constraints rather than by a second model, and candidates are scored by comparing answer sets rather than by asking a model. That leaves two model calls per question, the budget the UniQGen paper reports. Aggregates, `ORDER BY`, and `LIMIT` are not handled. A gold-oracle variant, which places reference answers in the context, is configured for ablation and is not a result.

Stores expose one method, `query`, returning SPARQL 1.1 JSON. The same system therefore runs on local RDF files in Oxigraph [8] (the default) or rdflib, or on any SPARQL endpoint. A cache wraps the store, memoizes results, and counts executions. The model client is likewise cached on disk, so a rerun replays the same completions. The default model is a local Ollama model; a config may name any OpenAI-compatible endpoint instead.

An experiment is a YAML file: benchmark, system, model, and optional language filter, item limit, and few-shot pool. A run predicts,

Table 4: Normalized benchmarks. Question counts are source questions; Text2SPARQL items are scored per language.

Benchmark	Questions	Graph
CK26	50	Corporate KG, local
DB26	50	DBpedia, organizers' endpoint
CK25	50	Corporate KG, local
DB25	100	DBpedia
QALD-10 [4]	394	Wikidata, live
QALD-9-plus [5]	150	DBpedia, live

executes, and scores each item, and can resume after an interruption. It writes the config, per-item predictions and traces, aggregate metrics, and a provenance record: model, model calls, cache hits, graph executions, git commit, and wall time. The same process can be served on the Text2SPARQL challenge API, so a system can be submitted to the organizers' client.

5 BENCHMARKS, METRICS, AND CAVEATS

Table 4 is the normalized suite. Each benchmark is pinned to a source commit and a checksum. Questions carry an id of the form used by the official scorer, a language, the gold query, feature tags (`COUNT`, `GROUP`, `ORDER`, `ASK`, and the rest), and whether order matters. Gold answers are stored in the official layout: every value of every variable flattened into one set. QALD answers are converted into that layout. Adapters isolate the source formats, so a new benchmark is a loader plus a manifest.

The scorer follows the Text2SPARQL pipeline [1, 7]. Every binding value is a document of relevance 1. Per question it reports set precision, set recall, and set F_1. Where the organizers mark order as significant it also reports nDCG. The headline `set_F_ndcg` is nDCG on those questions and set F_1 elsewhere. A checker rescores every published Text2SPARQL 2026 prediction; the project reports that all 5,024 per-question and average numbers match the official files. Runs report three aggregates side by side, because they answer different questions: the official metrics on every item, a strict variant, and the official metrics on the items whose gold query reproduces on our store.

Several properties of the data change how a number should be read.

(1) Forty-nine of the fifty CK26 gold queries appear verbatim in CK25, on the same graph. Neither edition is a development set for the other, and few-shot examples drawn from one leak the other. Runs record that overlap as contaminated examples. DB25 and DB26 share no gold query.
(2) Official ASK scoring is degenerate. The scorer treats an ASK answer as a document named `true`. A gold `true` scores 1.0 whatever the system returns; a gold `false` scores 0.0 for every system. The strict variant scores Booleans by exact match.
(3) Official nDCG ignores result order. Every returned value has the same score, so trec_eval orders values by their text. The strict variant uses the order the system returned.
(4) Official `set_F_ndcg` counts the mean nDCG of a scope as one extra item in the average. Hermes reproduces that, so its official column matches the leaderboard. The strict column is a plain mean.
(5) CK gold answers were computed on Tentris. On Oxigraph, 43 of 50 CK26 gold queries return the gold answers. The other seven break ties under `LIMIT` differently, format decimals differently, or call `xsd:int()`, which is not a SPARQL 1.1 function. Those items are flagged `gold_reproduces`. On rdflib, 42 of 50 agree. On the organizers' DBpedia endpoint, all 50 DB26 gold queries agree.
(6) QALD-9-plus targets DBpedia 2016-10, and QALD-10 was answered on Wikidata in 2022. The live endpoints have changed since, so a score against today's endpoint is not a score against the original graph.

Table 5: Finished CK26 runs, official metrics except the strict F_1 column. Model: `llama3.1`. Published Text2SPARQL entries are not in this table; they were scored by the organizers on Tentris.

System	F_1	P	R	F_{ndcg}	strict F_1
training-free	0.017	0.036	0.015	0.018	0.017
uniqgen	0.064	0.061	0.142	0.064	0.064

Comparable numbers follow from these facts. Official metrics on all items are what can be set next to published entries. Strict and reproducible figures stay next to them. A run with `leak_gold` or with contaminated examples is an upper bound. The model is part of the result: several published entries used larger API models, and a run on local `llama3.1` is a lower bound for a method.

6 PRELIMINARY RESULTS

Two runs have finished, both on CK26 (50 questions in English and German, 100 items), both with local Ollama `llama3.1`, and both with `contaminated_examples` at 0. Table 5 rounds the run files to three decimal places. The one-shot system made 116 model calls and 136 graph executions (373 s wall, median latency 3.0 s, 95th percentile 8.5 s) and failed to produce a runnable query on 24 items. UniQGen, at beam width 5, made 198 model calls and 667 graph executions (2,136 s wall, median 15.1 s, 95th percentile 58.1 s) and failed on 41 items. On the 86 items whose gold query reproduces, `set_F_ndcg` is 0.022 for the one-shot system and 0.065 for UniQGen.

Table 6 is the published CK26 leaderboard, for scale. It is not a paired comparison. The organizers scored those systems on Tentris, the model behind each entry is not controlled here, and our two runs execute on Oxigraph. Read against that board, both local runs are an order of magnitude below the strongest published entry and below the weakest of them. Between our two runs, the constraint search returns higher recall than the one-shot draft and spends more calls and more executions to do it. That is a fact about these two configurations on this model. It is not yet evidence about operator-level feedback: the algebra agent and the prover have no finished run, and the gold-oracle ablation has not been run. No DB26, QALD, or model-scaling number is included.

Table 6: Published Text2SPARQL 2026 CK26 scores (`set_F_ndcg`), organizer evaluation.

Entry	F_{ndcg}
SPARQL-LLM	0.832
INFAI-ETI-AND-FRIENDS-A	0.633
ADFR	0.513
INFAI-ETI-AND-FRIENDS-C	0.452
LIBER-AI-CLAUDE	0.335
IRIS	0.217
LIBER-AI-QWEN	0.206
INFAI-ETI-AND-FRIENDS-B	0.157

7 CONCLUSION

Hermes makes natural language to SPARQL a planning problem whose actions are the SPARQL algebra and whose feedback is execution. A Lean-style kernel types each step against the schema and holds the goal open until the plan discharges it. The one-shot baseline and UniQGen already run behind that interface, on benchmarks whose official scores the scorer reproduces, with the ASK, nDCG, overlap, and engine caveats kept visible beside the official column. The first CK26 runs, on `llama3.1`, land at 0.018 and 0.064 `set_F_ndcg`. The measurements that would speak to the four questions in the introduction — the algebra agent, the prover, the feedback ablations, search against the prompted policy, and the move to DBpedia and Wikidata — are still to be run.

REFERENCES

[1] AKSW. 2026. The Text2SPARQL Challenge. Challenge site and result repository. https://github.com/AKSW/text2sparql.aksw.org

[2] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction — CADE 28 (LNCS)*, Vol. 12699. Springer, 625–635.

[3] Alin Deutsch, Lucian Popa, and Val Tannen. 1999. Physical Data Independence, Constraints, and Optimization with Universal Plans. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*. 459–470.

[4] KGQA. 2022. QALD-10: Question Answering over Linked Data. Dataset, MIT license. https://github.com/KGQA/QALD-10

[5] KGQA. 2022. QALD-9-plus. Dataset, CC BY 4.0. https://github.com/KGQA/QALD_9_plus

[6] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going? arXiv:2408.05109. https://arxiv.org/abs/2408.05109

[7] National Institute of Standards and Technology. 2024. trec_eval. Information retrieval evaluation software. https://github.com/usnistgov/trec_eval

[8] Oxigraph contributors. 2026. Oxigraph. RDF graph database. https://github.com/oxigraph/oxigraph

[9] W3C RDF Working Group. 2014. RDF Schema 1.1. W3C Recommendation, 25 February 2014. https://www.w3.org/TR/rdf-schema/

[10] W3C SPARQL Working Group. 2013. SPARQL 1.1 Query Language. W3C Recommendation, 21 March 2013. https://www.w3.org/TR/sparql11-query/

[11] Mengying Wang, Nicolaas Jedema, Rahul Pandey, RaviKiran Krishnan, Jens Lehmann, and Yinghui Wu. 2026. Graph Query Generation with Constraint-guided Large Language Agents. arXiv:2605.00845. https://arxiv.org/abs/2605.00845